XSS Attack and Defense

Liran Ma and Zhipeng Cai
Department of Computer Science
Texas Christian University

Copyright © 2021 Liran Ma, Texas Christian University.
The development of this document is partially funded by the National Science Foundation's and SaTC program under award No. DGE-1912755. and Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.2 or any later version published by the Free Software Foundation. A copy of the license can be found at http://www.gnu.org/licenses/fdl.html.

Contents

1 Stored XSS Attack **1**

2 XSS Reflection Attack **4**

3 General Questions **6**

1 Stored XSS Attack

Cross-Site Scripting (XSS) attacks are a type of injection, in which malicious scripts are injected into otherwise benign and trusted websites. Stored (also known as second-order or persistent XSS) attacks, the injected script is permanently stored on the target servers, such as in a database, in a message forum, visitor log, and comment field. The victim then retrieves the malicious script from the server when it requests the stored information.

1.1 Lab Prep

Pull the lab related configuration files from the GitHub repository.

```
# Make sure that you are in the Docker_Security_Labs folder
# Use the pwd command can verify your current folder
git clone https://github.com/liranmatcu/Docker_Security_Labs.git
# or ``git pull'' if you have cloned the repository previously
```

Go to the xss_flask folder under Docker_Security_Labs as follows,

```
cd  Docker_Security_Labs/xss_flask/
```

and start container instances for this lab using docker-compose.

```
docker-compose  up   -d
# Or docker compose  up
```

The docker-compose command will start a container named xss-flask, which runs a website that allows users to submit comments on blog posts. You can view its detailed information using the "*docker ps*" command.

1.2 Launch the Attack

To start, open a browser and go to http://127.0.0.1:5000/blog (if the xss-flask container in running on your local machine; otherwise you need to replace 127.0.0.1 with the right IP address) to see the sample blog application. You can hit "Add Blog" and it allows clients to insert data such as texts or javascript code (as shown in Fig. 1).

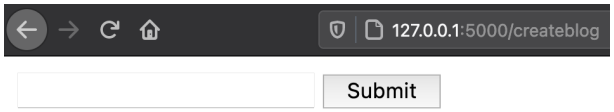
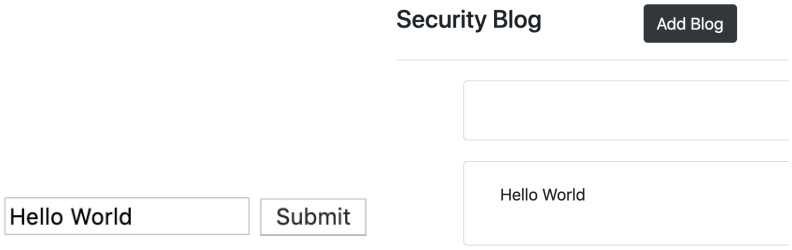


Figure 1: Create a blog.

On the web server, there are no protections against scripting. An attacker's target is to steal the personally identifiable information about legitimate users and their session data (e.g., cookies). Below are some examples of how to embed custom data/code onto the web application.

Non Malicious Post

First, we will submit a non-malicious post as shown in Fig. 2. A user can type "Hello World" and hit submit. After this comment has been submitted, any user who visits the blog post will receive "Hello World" within the application's response.

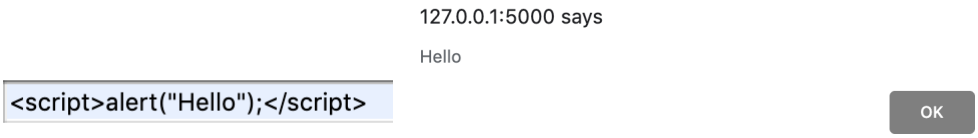


(a) Input data. (b) Output.

Figure 2: Non malicious post.

Non Malicious Injection Attack

We next will show an example of a non-malicious injection attack, where javascript (JS) code can be injected. In the example shown in Fig. 3, *<script>alert("Hello");</script>* is injected to the site. after the JS code is injected and stored on the web site, each user view the page will receive the pop-up message "Hello".



(a) Input JS code. (b) Output.

Figure 3: Non malicious injection.

Malicious Injection Attack

In Fig. 4, *<script>alert(document.cookie);</script>* is injected in order to display (eventually, steal) the cookies of a legitimate user.

Figure 4: Malicious injection data.

Fig. 5 show the results of the injected code.

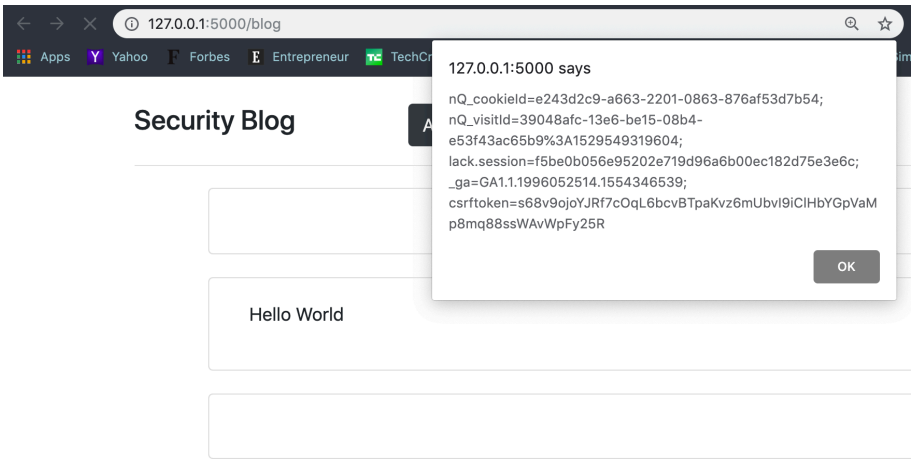


Figure 5: Malicious injection results.

1.2.1 Explain what can the malicious injection attack achieve? Why such attack is possible?

1.2.2 Defend the attack. You should now see the dangers of stored XSS and want to defend your website. The next portion involves diving into the python code to prevent these kind of attacks. The goal is to stop the same attacks you were running in the previous section. Hint: Go into the app.py file and uncomment the two lines in the blogcreate function (use vim on docker container). Then restart the docker container. The existing attacks will still run but any additional ones will not execute.

2 XSS Reflection Attack

Reflected attacks are those where the injected script is reflected off the web server, such as in an error message, search result, or any other response that includes some or all of the input sent to the server as part of the request. Reflected attacks are delivered to victims via another route, such as in an e-mail message, or on some other website. When a user is tricked into clicking on a malicious link, submitting a specially crafted form, or even just browsing to a malicious site, the injected code travels to the vulnerable web site, which reflects the attack back to the user's browser. The browser then executes the code because it came from a "trusted" server. Reflected XSS is also sometimes referred to as Non-Persistent or Type-II XSS.

First, start up the hacking server to explore a reflection attack. Log onto the web server as follows.

```
docker exec -it xss-flask sh
```

Next, issue the following command to start a hacking server listening on port 5001 (as shown in Fig. 6).

```
python3 hacking.py
```

This newly started web server will take the "reflected" information from a legitimate user.

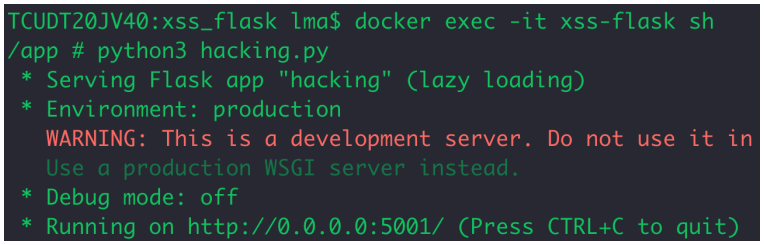
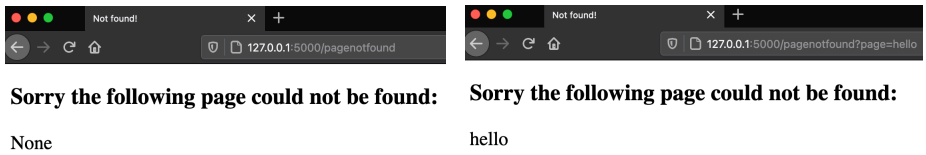


Figure 6: Start the reflected web server.

Navigate to http://127.0.0.1:5000/pagenotfound to begin this lab (as shown in Fig. 7a). This page takes in a get request from the url argument page. It can display content supplied by

the url argument. For example http://127.0.0.1:5000/pagenotfound?page=hello will show "hello" in the not found page (as shown in Fig. 7b).

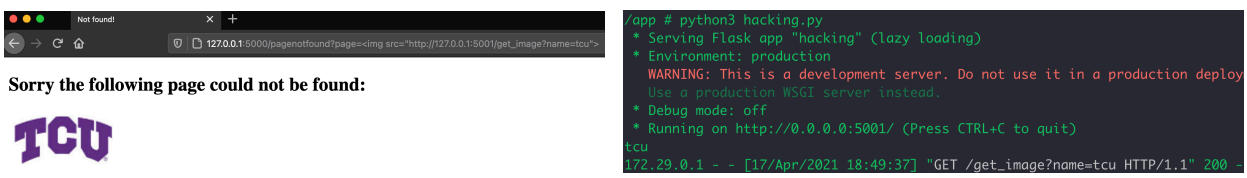


(a) Input JS code. (b) Output.

Figure 7: Non malicious injection.

To manipulate the site, one need to inject code into the get request of the url. This means the url should look something like http://127.0.0.1:5000/pagenotfound?page=your-code-here. You should be able to manipulate the browser into performing dangerous actions.

For example, we can use an image to execute code. Type following URL: http://127.0.0.1:5000/pagenotfound?page=<imgsrc="http://127.0.0.1:5001/get_image?name=tcu">, and it would display an image form the hacking web server started in the beginning of this section (as shown in Fig. 8a). The hacking web server also shows that a GET request was received (as shown in Fig. 8b).



(a) Display an image on the non found page. (b) Hacking server debugging information.

Figure 8: Display an image from an outside server.

If one wants to steal the cookies, the url could be supplied as follows: http://127.0.0.1:5000/pagenotfound?page=<script>alert(document.cookie);</script>. Due to the protection mechanisms (e.g., X-XSS-Protection and Content-Security-Policy) enforced by modern browsers, you may not able to to steal the cookies. When code in a request or url is detected, modern browsers will not execute it on the page.

2.0.1 With the current measures in place to defend what do you think attackers will do in the future? How will IT departments and developers have to respond to this new threat?

3 General Questions

3.0.1 Any corrections or general comments about this lab?

3.0.2 How long did it take you to complete this lab?

3.0.3 Was it an appropriate length lab?